TVQ / BÀI THỰC HÀNH 3

SPI & I²C

Hướng dẫn từng thao tác, mã nguồn và mẫu ghi kết quả

Bản soạn riêng từ yêu cầu môn học; không dùng số liệu của tác giả repo làm kết quả của bạn.

Trong tập này

3.1 · Đọc DS1307 qua I²C

3.2 · Đồng hồ báo thức bốn nút

3.3 · Bốn MAX7219 chạy chữ qua SPI

Cách dùng

1. Xem sơ đồ và chuẩn bị phần cứng. 2. Mở đúng tệp code trong danh sách. 3. Làm từng bước, chụp kết quả thực. 4. Đối chiếu tiêu chí và trả lời câu hỏi. Các tệp cấu hình có placeholder cần điền trước khi chạy.

Các mã Arduino/ESP và kịch bản GUI cần được biên dịch/nạp/chạy trên máy, board của bạn. Báo cáo kiểm tra PC nằm trong docs/KIEM_THU.md. Không hiểu "có code" là đã kiểm thử phần cứng.

Nguồn đối chiếu

BTH3_SPI_I2C.pdf trong tai-lieu-goc của github.com/buncric/truyenthongiot (commit 7a7f6c3). Nội dung hướng dẫn/mã mới; tên, địa chỉ và topic ví dụ TVQ. Tài liệu kỹ thuật chính thức liệt kê ở docs/NGUON.md.

BÀI 3.1

Đọc DS1307 qua I²C

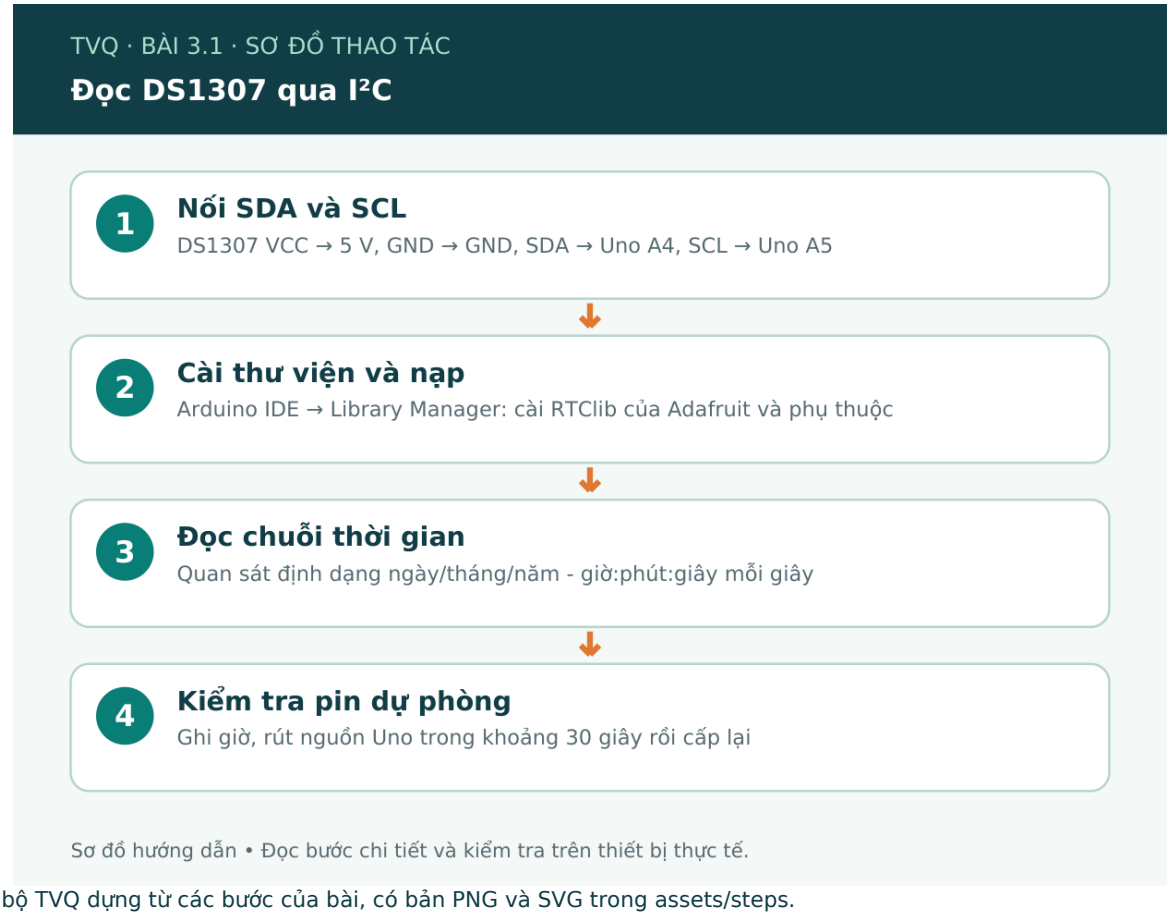
Đọc ngày giờ theo chu kỳ và giữ thời gian khi ngắt nguồn chính.

Chuẩn bị

Uno, module DS1307 5 V, pin dự phòng đúng loại module; thư viện RTClib.

Mở những tệp này

code/rtc_serial/rtc_serial.ino



Ảnh do bộ TVQ dựng từ các bước của bài, có bản PNG và SVG trong assets/steps.

3.1 / THAO TÁC CHI TIẾT

1. Nối SDA và SCL

DS1307 VCC → 5 V, GND → GND, SDA → Uno A4, SCL → Uno A5. Kiểm tra ký hiệu in trên module. Cấu hình này chọn Uno 5 V; nếu dùng ESP 3,3 V cần xét pull-up và chuyển mức, không chép nguyên dây 5 V sang ESP.

2. Cài thư viện và nạp

Arduino IDE → Library Manager: cài RTClib của Adafruit và phụ thuộc. Nạp rtc_serial cho Uno. Mở Serial Monitor 115200. Nếu RTC chưa chạy, code đặt giờ theo thời điểm biên dịch; nếu đang chạy thì giữ nguyên.

3. Đọc chuỗi thời gian

Quan sát định dạng ngày/tháng/năm - giờ:phút:giây mỗi giây. millis() lập lịch, RTC cung cấp thời gian. Nếu giờ khác thực tế, cần đặt giờ RTC một lần bằng rtc.adjust(DateTime(...)) rồi gỡ lệnh đặt cưỡng bức sau khi nạp.

4. Kiểm tra pin dự phòng

Ghi giờ, rút nguồn Uno trong khoảng 30 giây rồi cấp lại. Nếu có pin phù hợp, RTC tiếp tục đếm trong thời gian mất nguồn. Ghi kết quả quan sát, không coi thời gian biên dịch là giờ hiện tại chính xác.

Đạt yêu cầu khi

- Serial tăng giây đều.
- Reset Uno không tự đặt lại một RTC đang chạy.
- Chênh lệch sau mất nguồn được ghi lại.

Giải thích ngắn

I²C dùng địa chỉ để làm gì?

Master chọn slave trên chung SDA/SCL; mỗi thiết bị cần địa chỉ phù hợp, DS1307 thường 0x68.

Pin RTC cấp cho Uno không?

Không. Pin giữ mạch đồng hồ, không chạy toàn bộ Uno và LCD.

Khi gặp lỗi

- RTC missing: kiểm tra A4/A5, nguồn và địa chỉ.
- Giờ đứng: kiểm tra oscillator/module/pin và trạng thái isrunning.

Minh chứng cần bổ sung: ảnh thao tác thật, log/capture/project phù hợp và bảng kết quả do bạn đo. Không điền số trong ảnh tham khảo thành kết quả của mình.

BÀI 3.2

Đồng hồ báo thức bốn nút

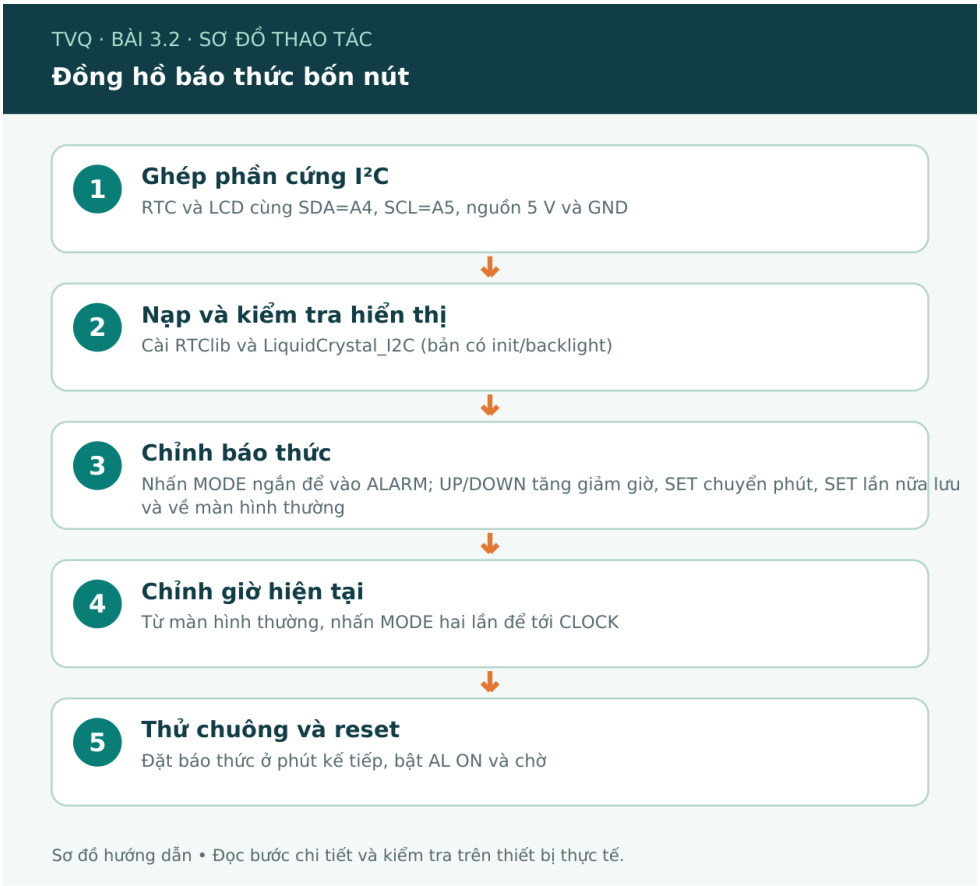
Hiển thị thời gian, chỉnh báo thức/giờ RTC bằng nút và lưu báo thức trong EEPROM.

Chuẩn bị

Uno, DS1307, LCD I²C 16×2, buzzer thụ động D8, bốn nút D2/D3/D4/D5–GND.

Mở những tệp này

code/alarm_clock/alarm_clock.ino



Ảnh do bộ TVQ dựng từ các bước của bài, có bản PNG và SVG trong assets/steps.

3.2 / THAO TÁC CHI TIẾT

1. Ghép phần cứng I²C

RTC và LCD cùng SDA=A4, SCL=A5, nguồn 5 V và GND. LCD mặc định 0x27, RTC 0x68 nên có thể dùng cùng bus. Buzzer nhỏ thụ động nối D8/GND phù hợp dòng chân; tải lớn dùng transistor. Nút lần lượt MODE, UP, DOWN, SET ở D2–D5.

2. Nạp và kiểm tra hiển thị

Cài RTClib và LiquidCrystal_I2C (bản có init/backlight). Nạp alarm_clock. Chỉnh biến trở tương phản LCD. Dòng đầu là giờ, dòng hai AL HH:MM ON/OFF. Nếu LCD địa chỉ 0x3F thì đổi constructor trong code.

3. Chỉnh báo thức

Nhấn MODE ngắn để vào ALARM; UP/DOWN tăng giảm giờ, SET chuyển phút, SET lần nữa lưu và về màn hình thường. Giữ MODE 2 giây để bật/tắt AL. EEPROM chỉ được ghi khi lưu/toggle, không ghi mỗi vòng lặp.

4. Chỉnh giờ hiện tại

Từ màn hình thường, nhấn MODE hai lần để tới CLOCK. UP/DOWN chỉnh giờ rồi SET chuyển phút; SET lần nữa đặt RTC với ngày hiện có và giây 0. Bản này chỉnh giờ/phút, không cung cấp menu chỉnh ngày.

5. Thử chuông và reset

Đặt báo thức ở phút kế tiếp, bật AL ON và chờ. Chuông tắt khi bấm nút bất kỳ hoặc sau 30 giây. Code khóa một lần trong cùng phút để tránh kêu lại ngay sau khi tắt. Reset Uno để xác nhận giờ/phút báo thức còn trong EEPROM.

Đạt yêu cầu khi

- Cả bốn nút hoạt động và LCD cập nhật.
- Chuông không kích hoạt lại nhiều lần trong một phút.
- EEPROM giữ báo thức sau reset.

Giải thích ngắn

Vì sao dùng millis thay delay dài?

Nút vẫn được quét và RTC/LCD được cập nhật trong lúc chờ, tránh chặn toàn bộ vòng lặp.

RTC và EEPROM khác nhau?

RTC giữ thời gian; EEPROM lưu cấu hình báo thức/trạng thái khi mất nguồn.

Khi gặp lỗi

- LCD sáng nhưng không chữ: chỉnh tương phản và địa chỉ.
- Chuông liên tục: kiểm tra loại buzzer và thao tác tắt.

Minh chứng cần bổ sung: ảnh thao tác thật, log/capture/project phù hợp và bảng kết quả do bạn đo. Không điền số trong ảnh tham khảo thành kết quả của mình.

BÀI 3.3

Bốn MAX7219 chạy chữ qua SPI

Hiển thị TVQ - IOT trên ma trận 8×32 và nhận biết chuỗi cascade.

Chuẩn bị

Uno, cụm bốn MAX7219 loại FC-16, nguồn 5 V đủ dòng, MD_Parola + MD_MAX72XX.

Mở những tệp này

code/matrix_spi/matrix_spi.ino



Ảnh do bộ TVQ dựng từ các bước của bài, có bản PNG và SVG trong assets/steps.

3.3 / THAO TÁC CHI TIẾT

1. Nối đầu vào cụm

VCC → 5 V đủ dòng, GND chung; DIN → Uno D11/MOSI, CLK → D13/SCK, CS/LOAD → D10. Nối vào đầu IN của module đầu tiên. Nếu bốn module rời: DOUT module trước → DIN module sau; CLK và CS dùng chung.

2. Cài và biên dịch

Cài MD_Parola, MD_MAX72XX trong Library Manager. Mở matrix_spi, chọn Uno rồi Verify/Upload. Code khai báo 4 module và loại FC16_HW; nếu module khác phải chọn loại phù hợp.

3. Quan sát cuộn chữ

Dòng TVQ - IOT cuộn ngang. setIntensity(2) giữ độ sáng ban đầu thấp. Không lấy nguồn LED công suất lớn trực tiếp từ chân GPIO. Khi chỉ một module sáng, kiểm tra chiều dây cascade.

4. Thử thay đổi

Sửa chuỗi thành tên riêng không dấu, tăng/giảm tốc độ 70 ms trong displayText. Ghi cảm nhận tốc độ và ảnh toàn cụm 8×32. Trong Proteus cần có đúng model và HEX; bộ này cung cấp mã, không có project Proteus đã chạy.

Đạt yêu cầu khi

- Đủ bốn module hiển thị.
- Ký tự không bị đảo do sai loại module.

Giải thích ngắn

Vì sao không dùng MISO?

MAX7219 nhận lệnh một chiều trong bài; dữ liệu đi qua DIN/DOUT của chuỗi, không có phản hồi SPI về Uno.

CS làm gì?

Chốt dữ liệu dịch vào các thanh ghi theo giao dịch; số IC trong chuỗi quyết định lượng dữ liệu cần gửi.

Khi gặp lỗi

- Chữ đảo: đổi hardware type theo module.
- Nhấp nháy/reset: kiểm tra nguồn và GND.

Minh chứng cần bổ sung: ảnh thao tác thật, log/capture/project phù hợp và bảng kết quả do bạn đo. Không điền số trong ảnh tham khảo thành kết quả của mình.

PHỤ LỤC / SƠ ĐỒ NỐI DÂY

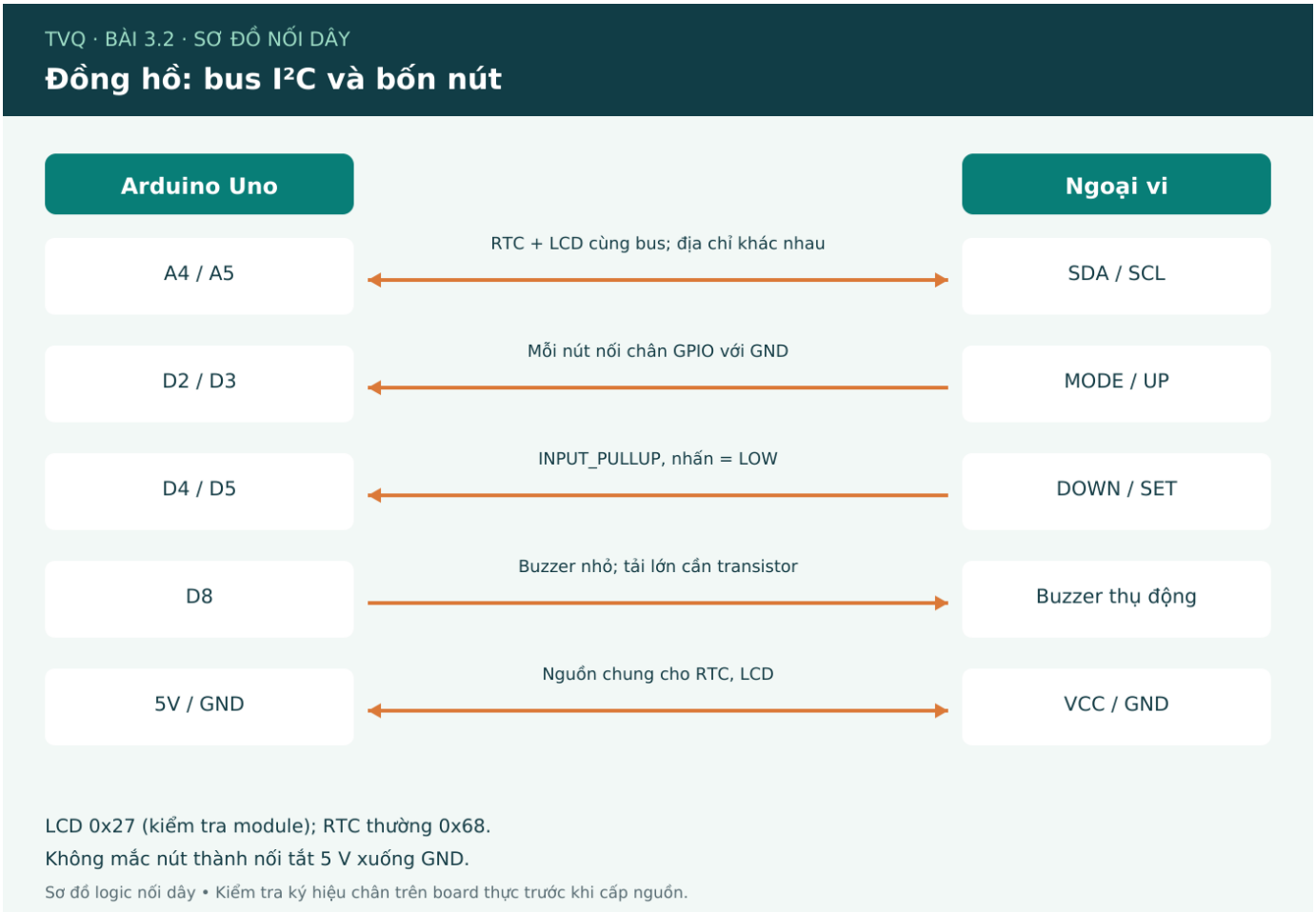
Đối chiếu chân trong code và sơ đồ trước khi cấp nguồn. Hình thể hiện kết nối logic, không phải ảnh đo phần cứng.



Bản PNG/SVG trong assets/wiring; ký hiệu trên module thực tế là căn cứ cuối cùng. Sơ đồ S3 LED ngoài chỉ dùng cho biến thể ESP32-S3, Wemos dùng LED onboard GPIO2 active-low.

PHỤ LỤC / SƠ ĐỒ NỐI DÂY

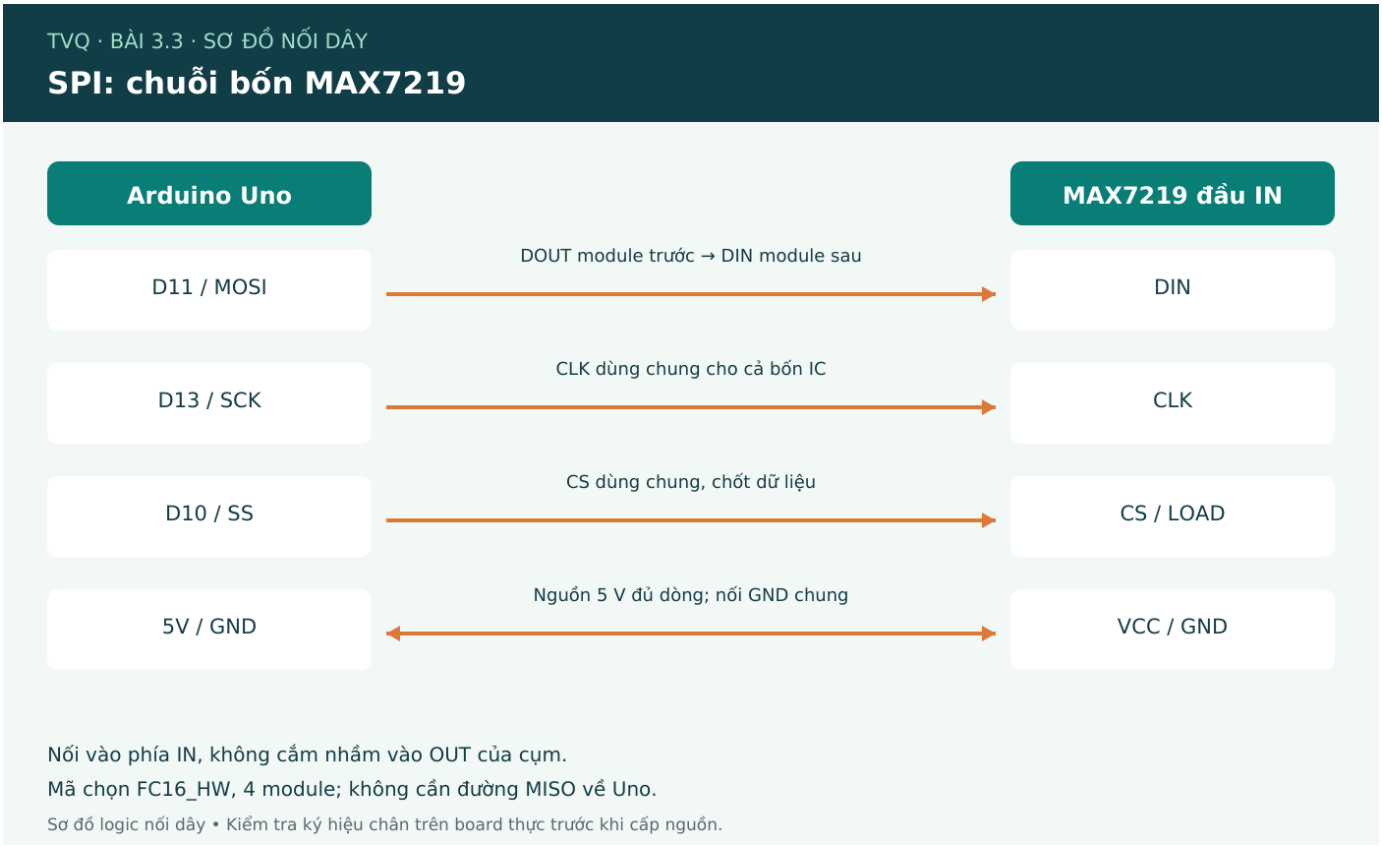
Đối chiếu chân trong code và sơ đồ trước khi cấp nguồn. Hình thể hiện kết nối logic, không phải ảnh đo phần cứng.



Bản PNG/SVG trong assets/wiring; ký hiệu trên module thực tế là căn cứ cuối cùng. Sơ đồ S3 LED ngoài chỉ dùng cho biến thể ESP32-S3, Wemos dùng LED onboard GPIO2 active-low.

PHỤ LỤC / SƠ ĐỒ NỐI DÂY

Đối chiếu chân trong code và sơ đồ trước khi cấp nguồn. Hình thể hiện kết nối logic, không phải ảnh đo phần cứng.



Bản PNG/SVG trong assets/wiring; ký hiệu trên module thực tế là căn cứ cuối cùng. Sơ đồ S3 LED ngoài chỉ dùng cho biến thể ESP32-S3, Wemos dùng LED onboard GPIO2 active-low.